Leme

Manual técnico

Instalação, modo interativo e modo direto, referência de cada comando, perfis, scripts e códigos de saída, exportação e importação, diagnóstico, console web, API REST e integrações, operação do cluster, segurança, desempenho e solução de problemas da CLI do cluster chave-valor.

Versão 1.1.0 · Outubro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

01 Apresentação 5

1.1 Novidades da 1.1.0 5
1.2 O que vem no projeto 5
1.3 Tecnologias 6
1.4 Estrutura de pastas 7
1.5 Como ler este manual 7

02 Instalação 8

2.1 Requisitos 8
2.2 Com Docker 8
2.3 Sem Docker 9
2.4 Binários prontos 9
2.5 Primeiro acesso 9
2.6 Autocompletar do shell 10

03 Conceitos 11

3.1 Cluster e nós 11
3.2 Anel e réplicas 11
3.3 Consistência 11
3.4 Versões 11
3.5 TTL e lápides 12
3.6 Dicas e reparo 12

04 Conectando ao cluster 13

4.1 Informando os nós 13
4.2 Perfis de conexão 13
4.3 Ordem de prioridade 14
4.4 Token 14
4.5 TLS 14

05 Modo interativo 15

5.1 Abrindo 15
5.2 Editando a linha 16
5.3 Valores com espaços e caracteres especiais 16
5.4 Comandos de sessão 16
5.5 Confirmações 17

Copyright (©) 2026 Erik Pimenta

06 Scripts e automação 18

6.1 Modo direto 18
6.2 Formatos de saída 18
6.3 Códigos de saída 18
6.4 Entrada padrão e arquivos 19
6.5 Arquivos de comandos 19
6.6 PowerShell 19
6.7 Backup lógico e migrações 20

07 Referência de comandos 21

7.1 get 21
7.2 put 21
7.3 del 21
7.4 exists, ttl e expire 22
7.5 scan, keys e count 22
7.6 incr e decr 22
7.7 copy e rename 22
7.8 mget e mset 23
7.9 export e import 23
7.10 locate e ping 24
7.11 cluster status, nodes e ring 24
7.12 node stats, compact e leave 24
7.13 doctor 24
7.14 watch e top 25
7.15 Ferramentas 26

08 Console web 27

8.1 Abrindo 27
8.2 Organização da tela 27
8.3 Visão geral 28
8.4 Nós e anel 28
8.5 Chaves 29
8.6 Terminal 30
8.7 Diagnóstico 30
8.8 Integrações 31
8.9 Paleta de comandos e atalhos 32
8.10 Tema, celular e acessibilidade 33

09 API REST e integrações 34

9.1 Autenticação 34
9.2 Rotas 34
9.3 Kubernetes 35
9.4 systemd 35
9.5 Prometheus e Grafana 36

Copyright (©) 2026 Erik Pimenta

10 Operando o cluster 37
10.1 O nó lemed 37
10.2 Subindo um cluster 38
10.3 Como serviço do systemd 38
10.4 Crescendo o cluster 38
10.5 Retirando um nó 38
10.6 Falhas e recuperação 39
10.7 Compactação 39
10.8 Backup 39
10.9 Monitoramento 40
11 Segurança 41
11.1 Token 41
11.2 Certificados para TLS e mTLS 41
11.3 Console e API 41
11.4 Boas práticas 42
12 Desempenho e escala 43
12.1 Medindo 43
12.2 O que ajustar 43
12.3 Limites 43
13 Testes, build e CI 45
13.1 Comandos 45
13.2 O que os testes cobrem 45
13.3 Integração contínua 45
14 Solução de problemas 46
14.1 Conexão 46
14.2 Dados 46
14.3 Terminal 47
15 Licenças 48
15.1 Componentes de terceiros 48
15.2 Licença do Leme 48

Copyright (©) 2026 Erik Pimenta

CAPÍTULO **01**

Apresentação

O Leme é uma linha de comando para conversar com um cluster distribuído de armazenamento chave-valor. Ele tem dois jeitos de trabalhar: um terminal interativo, em que você se conecta uma vez e executa vários comandos, e um modo direto, em que cada chamada executa um comando e sai, pronto para scripts e automação.

1.1 Novidades da 1.1.0

A versão 1.1.0 amplia a CLI com ferramentas de dados e de diagnóstico, redesenha o console web e abre o Leme para integrações. Tudo o que existia na 1.0.0 continua funcionando do mesmo jeito.

ÁREA	O QUE CHEGOU
Dados	`incr` e `decr` (contadores atômicos), `copy` e `rename` preservando o TTL, `mget` e `mset` para várias chaves em paralelo
Backup lógico	`export` e `import` em NDJSON, JSON ou CSV, com TTL, versão e valores binários em base64
Operação	`doctor` com verificações e sugestões (novo código de saída 8), `watch` para acompanhar mudanças e `top`, um painel ao vivo no terminal
Console web	Nova identidade visual, barra lateral agrupada, faixa de status, sparklines, anel como mostrador, feed de eventos, paleta de comandos (`Ctrl+K`), Exportar e Importar, páginas Diagnóstico e Integrações
API REST	Rotas `/api/v1` documentadas em OpenAPI 3.1 e autenticação por `Authorization: Bearer`
Integrações	Arquivos prontos em `deploy/`: Kubernetes, systemd, Prometheus com regras de alerta e painel do Grafana

1.2 O que vem no projeto

PARTE	O QUE É
`leme`	A CLI. Modo interativo, modo direto, console web, cluster local de testes, carga sintética, diagnóstico, painel ao vivo, exportação e importação e perfis de conexão
`lemed`	O nó do cluster. Guarda as chaves em disco, replica as escritas e atende a CLI
Console web	Painel no navegador com visão geral, nós, anel, navegador de chaves, terminal, diagnóstico e integrações. Vem dentro do binário `leme`
API REST	As mesmas rotas que o console usa, documentadas em `/api/v1/openapi.json`, para scripts e outros sistemas
Docker Compose	Três nós e o console, com volumes e tokens gerados na primeira execução
Scripts iniciar	`iniciar.cmd` no Windows e `iniciar.sh` no Linux e no macOS: sobem tudo com um clique
Integrações	`deploy/`: StatefulSet do Kubernetes, unidade do systemd, configuração e alertas do Prometheus e painel do Grafana, também disponíveis no perfil `observabilidade` do Docker Compose

Copyright (©) 2026 Erik Pimenta

PARTE	O QUE É
Testes	Testes de unidade, testes de integração com clusters reais em memória e um teste de ponta a ponta com processos separados
Manuais	Este manual e o PDF de arquitetura do banco de dados

1.3 Tecnologias

CAMADA	TECNOLOGIA
Linguagem	Go 1.24, binários estáticos sem CGO
Linha de comando	Cobra e pflag
Rede	TCP com protocolo binário próprio, multiplexado, com TLS 1.2+ opcional e mTLS
Terminal	Editor de linha próprio, em modo cru, para Linux, macOS, BSD e Windows
Armazenamento	Log de registros com CRC32C, índice em memória e compactação
Console	HTML, CSS e JavaScript sem frameworks, embutidos com `go:embed`
API	HTTP e JSON, descrita em OpenAPI 3.1
Tipografia	Instrument Sans, Bricolage Grotesque e IBM Plex Mono
Observabilidade	`/saude` em JSON, `/metricas` no formato do Prometheus, regras de alerta e painel do Grafana
Implantação	Docker Compose, Kubernetes e systemd
Integração contínua	GitHub Actions

Copyright (©) 2026 Erik Pimenta

1.4 Estrutura de pastas

```
cmd/leme                  CLI
cmd/lemed                 nó do cluster
internal/cli              comandos Cobra, flags globais, perfis, dev, bench, console,
                          export, import, watch e top
internal/comandos         registro único dos comandos (CLI, REPL e terminal web),
                          diagnóstico, exportação e importação
internal/repl             modo interativo
internal/terminal         editor de linha e modo cru por sistema operacional
internal/saida            tabela, JSON e texto
internal/config           perfis de conexão
internal/cliente          cliente do cluster
internal/no               nó: coordenação, réplicas, gossip, dicas e métricas
internal/armazenamento    log, índice, recuperação e compactação
internal/anel             hash consistente
internal/protocolo        quadros e mensagens
internal/transporte       conexões, pool e TLS
internal/console          servidor, API REST e páginas do console web
deploy/kubernetes         StatefulSet, Service e Secret
deploy/systemd            unidade e arquivo de ambiente do lemed
deploy/prometheus         coleta e regras de alerta
deploy/grafana            painel pronto para importar
scripts                   teste de ponta a ponta e criação do .env no Windows
docs                      manuais e imagens
```

1.5 Como ler este manual

Os capítulos 2 a 4 levam você do zero a um cluster funcionando. Os capítulos 5 a 8 explicam o uso do dia a dia: o modo interativo, os scripts, a referência de cada comando e o console web. O capítulo 9 mostra a API REST e as integrações. Os capítulos 10 a 12 são para quem opera o cluster em produção. Comandos aparecem em `fonte mono`; o que muda de instalação para instalação aparece entre sinais de menor e maior, como `<chave>`.

> **Convenção dos exemplos**
>
> Os exemplos usam o binário como `leme`. No Windows, sem o binário no PATH, use `.\bin\leme.exe`.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 02

Instalação

O Leme roda de três formas: com Docker, que sobe três nós e o console; direto com o Go, usando o cluster local embutido; ou com os binários, em máquinas separadas. Para Kubernetes e systemd, veja o capítulo 9.

2.1 Requisitos

PARA RODAR COM	VOCÊ PRECISA	OBSERVAÇÃO
Docker	Docker com Docker Compose v2	Sobe 3 nós `lemed` e o console, cada nó com o seu volume
Go	Go 1.24 ou mais novo	Compila os dois binários em poucos segundos
Binários	Nada além do binário	Linux, Windows e macOS, em amd64 e arm64
Navegador	Chrome, Edge, Firefox ou Safari atuais	Só para o console web

2.2 Com Docker

01 Baixe o projeto pelo botão **Code › Download ZIP** do GitHub e descompacte, ou clone com `git clone https://github.com/ravelroot/Distributed-Key-Value-CLI.git`.

02 No Windows, dê dois cliques em `iniciar.cmd`. No Linux ou no macOS, abra um terminal na pasta e rode `./iniciar.sh`.

03 Na primeira vez, o script cria o arquivo `.env` a partir do `.env.example`, com um token aleatório para o cluster (`LEME_TOKEN`) e outro para o console (`LEME_TOKEN_CONSOLE`).

04 O script roda `docker compose up -d --build`, espera os nós ficarem saudáveis e abre o navegador no endereço do console, já com o token.

Comandos úteis depois da primeira vez:

COMANDO	O QUE FAZ
`docker compose up -d --build`	Sobe ou atualiza depois de mudar o código
`docker compose ps`	Mostra os contêineres e se estão saudáveis
`docker compose logs -f no1`	Acompanha os logs de um nó
`docker compose exec console leme --nos no1:7400`	Abre o modo interativo dentro da rede do cluster
`docker compose exec console leme --nos no1:7400 doctor`	Confere a saúde do cluster
`docker compose stop no3`	Derruba um nó para ver o cluster degradado
`docker compose --profile observabilidade up -d`	Sobe também o Prometheus e o Grafana (capítulo 9)
`docker compose down`	Para tudo, mantendo os dados
`docker compose down -v`	Para e apaga os volumes

Copyright (©) 2026 Erik Pimenta

> **Por que usar a CLI dentro do contêiner**
>
> Os nós se anunciam pelos nomes da rede do Docker (`no1:7400`). A CLI do seu computador também funciona com `--nos 127.0.0.1:7401,127.0.0.1:7402,127.0.0.1:7403`: ela percebe que os nomes internos não respondem e passa a coordenar pelas portas publicadas.

2.3 Sem Docker

Com o Go instalado:

```
git clone https://github.com/ravelroot/Distributed-Key-Value-CLI.git
cd Distributed-Key-Value-CLI
go build -o bin/ ./cmd/...
./bin/leme dev --console 127.0.0.1:8080
```

O `leme dev` sobe três nós no mesmo processo, nas portas 7401, 7402 e 7403, com os dados na pasta de configuração do usuário (`~/.config/leme/dev` no Linux, `%AppData%\leme\dev` no Windows). Os dados sobrevivem a reinícios; `--limpar` começa do zero. Em outro terminal:

```
./bin/leme --nos 127.0.0.1:7401
```

2.4 Binários prontos

Cada execução do CI publica os binários de Linux (amd64 e arm64), Windows (amd64) e macOS (arm64) como artefatos. Para compilar você mesmo para outra plataforma:

```
GOOS=windows GOARCH=amd64 CGO_ENABLED=0 go build -o dist/leme.exe ./cmd/leme
GOOS=linux GOARCH=arm64 CGO_ENABLED=0 go build -o dist/lemed ./cmd/lemed
```

O `Makefile` faz o mesmo com a versão e o commit gravados no binário: `make build`. Confira com `leme version`.

2.5 Primeiro acesso

01 Abra o modo interativo: `leme --nos 127.0.0.1:7401`.

02 A primeira linha mostra a versão; a segunda, o cluster, quantos nós estão vivos, o número de réplicas e a consistência.

03 Grave uma chave: `put usuario:1 '{"nome":"Ana"}'`.

04 Leia: `get usuario:1`. Para ver versão, TTL e réplicas: `get usuario:1 --detalhes`.

05 Veja o cluster: `cluster status` e `cluster nodes`. Para uma checagem completa, `doctor`.

06 Digite `ajuda` para a lista de comandos e `sair` (ou `Ctrl+D`) para encerrar.

Copyright (©) 2026 Erik Pimenta

2.6 Autocompletar do shell

O modo interativo já completa com `Tab`. Para o shell do sistema completar comandos e opções do modo direto:

SHELL	COMANDO
Bash	`leme completion bash > /etc/bash_completion.d/leme`
Zsh	`leme completion zsh > "${fpath[1]}/_leme"`
Fish	`leme completion fish > ~/.config/fish/completions/leme.fish`
PowerShell	`leme completion powershell | Out-String | Invoke-Expression` (adicione ao `$PROFILE`)

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 03

Conceitos

Este capítulo explica as ideias que aparecem nas saídas dos comandos. O PDF de arquitetura do banco de dados aprofunda cada uma.

3.1 Cluster e nós

Um cluster é um grupo de processos `lemed`, os nós, que juntos guardam as chaves. Todo nó tem um identificador (`no-1`), um endereço (`10.0.0.11:7400`) e um estado: `vivo`, `suspeito` (sem resposta há 3 segundos), `morto` (há 10 segundos) ou `saiu` (retirado com `node leave`). Os nós descobrem uns aos outros por gossip: a cada segundo, cada nó troca a lista de membros com os outros.

3.2 Anel e réplicas

As chaves são distribuídas por hash consistente. Cada nó ocupa 128 posições, os nós virtuais, num anel de 2^{64} posições. Uma chave cai numa posição e pertence aos N nós seguintes no sentido horário, as réplicas. A primeira é a primária. `leme locate <chave>` mostra quem são; `leme cluster ring` mostra a fatia do anel de cada nó.

3.3 Consistência

Cada operação escolhe quantas réplicas precisam responder:

CONSISTÊNCIA	ESCRITA CONFIRMA COM	LEITURA RESPONDE COM	QUANDO USAR
`um`	1 réplica	1 réplica	Menor latência, aceita ler um valor um pouco atrasado
`quorum` (padrão)	maioria (2 de 3)	maioria	Equilíbrio: tolera um nó fora e sempre lê a última escrita confirmada
`todos`	todas	todas	Máxima garantia; falha se qualquer réplica estiver fora

Com `quorum` nas leituras e nas escritas, as duas maiorias sempre se cruzam em pelo menos uma réplica, então toda leitura enxerga a última escrita confirmada.

3.4 Versões

Cada escrita recebe uma versão de 64 bits gerada pelo nó coordenador: os 48 bits de cima são o horário em milissegundos e os 16 de baixo, um contador. Quando duas réplicas discordam, vence a versão maior. `get --detalhes` mostra a versão e o horário da gravação; `put --se-versao` usa a versão para escritas condicionais, e `incr`, `decr` e `rename` se apoiam nelas.

Copyright (©) 2026 Erik Pimenta

3.5 TTL e lápides

Uma chave gravada com `--ttl 30m` expira sozinha em 30 minutos. Apagar uma chave grava uma lápide, uma marca versionada de que ela foi apagada, para que as réplicas atrasadas não a ressuscitem. Lápides e chaves expiradas saem do disco na compactação.

3.6 Dicas e reparo

Se uma réplica está fora do ar durante uma escrita, o coordenador guarda uma dica e entrega a escrita quando ela voltar. Se uma leitura percebe uma réplica atrasada, envia a versão nova para ela: é o reparo de leitura. `cluster status` mostra quantas dicas estão pendentes e quantos reparos foram feitos, e `doctor` avisa quando elas se acumulam.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 04

Conectando ao cluster

A CLI precisa de pelo menos um nó; ela descobre os outros sozinha. Informe vários para não depender de um só.

4.1 Informando os nós

```
leme --nos 10.0.0.11:7400,10.0.0.12:7400 cluster status
```

Sem `--nos`, a CLI usa a variável `LEME_NOS`, depois o perfil atual e, por último, `127.0.0.1:7400`.

4.2 Perfis de conexão

Um perfil guarda nós, token, TLS, consistência, tempo limite e formato de saída. O primeiro perfil criado vira o padrão.

```
leme config set local --nos 127.0.0.1:7401,127.0.0.1:7402
leme config set producao --nos 10.0.0.11:7400,10.0.0.12:7400 \
      --token-arquivo ~/.leme-token --tls-ca ca.pem --consistencia quorum --tempo-limite 3s
leme config list
leme config use producao
leme --perfil local get usuario:1
leme config view
```

COMANDO	O QUE FAZ
`config set <perfil>`	Cria ou altera; só as opções informadas mudam
`config use <perfil>`	Define o perfil padrão
`config list`	Lista, com * no padrão
`config view`	Mostra a conexão que será usada, sem revelar o token
`config delete <perfil>`	Apaga

O arquivo fica em `~/.config/leme/config.json` (Linux), `~/Library/Application Support/leme/config.json` (macOS) ou `%AppData%\leme\config.json` (Windows), com permissão de leitura só para o dono. `LEME_CONFIG` ou `--config` usam outro arquivo.

Copyright (©) 2026 Erik Pimenta

4.3 Ordem de prioridade

Para cada opção, vale a primeira que existir:

01 A flag na linha de comando (`--nos`, `--token`, `-o`, `-c`).

02 A variável de ambiente (`LEME_NOS`, `LEME_TOKEN`, `LEME_SAIDA`, `LEME_CONSISTENCIA`, `LEME_PERFIL`).

03 O perfil escolhido com `--perfil`, ou o perfil padrão.

04 O valor de fábrica.

4.4 Token

Se o cluster exige token, a CLI envia o token logo depois de conectar. Prefira `--token-arquivo` ou a variável `LEME_TOKEN` a `--token`, que fica no histórico do shell. Um token errado encerra o comando com o código 7.

4.5 TLS

Com nós em TLS, informe a CA que assinou os certificados deles:

```
leme --nos no1.interno:7400 --tls-ca ca.pem cluster status
```

Se os nós exigem certificado do cliente (mTLS), acrescente `--tls-cert cliente.pem --tls-chave cliente.key`. `--tls-inseguro` aceita qualquer certificado e serve só para testes. O capítulo 11 mostra como gerar os certificados.

Copyright (©) 2026 Erik Pimenta

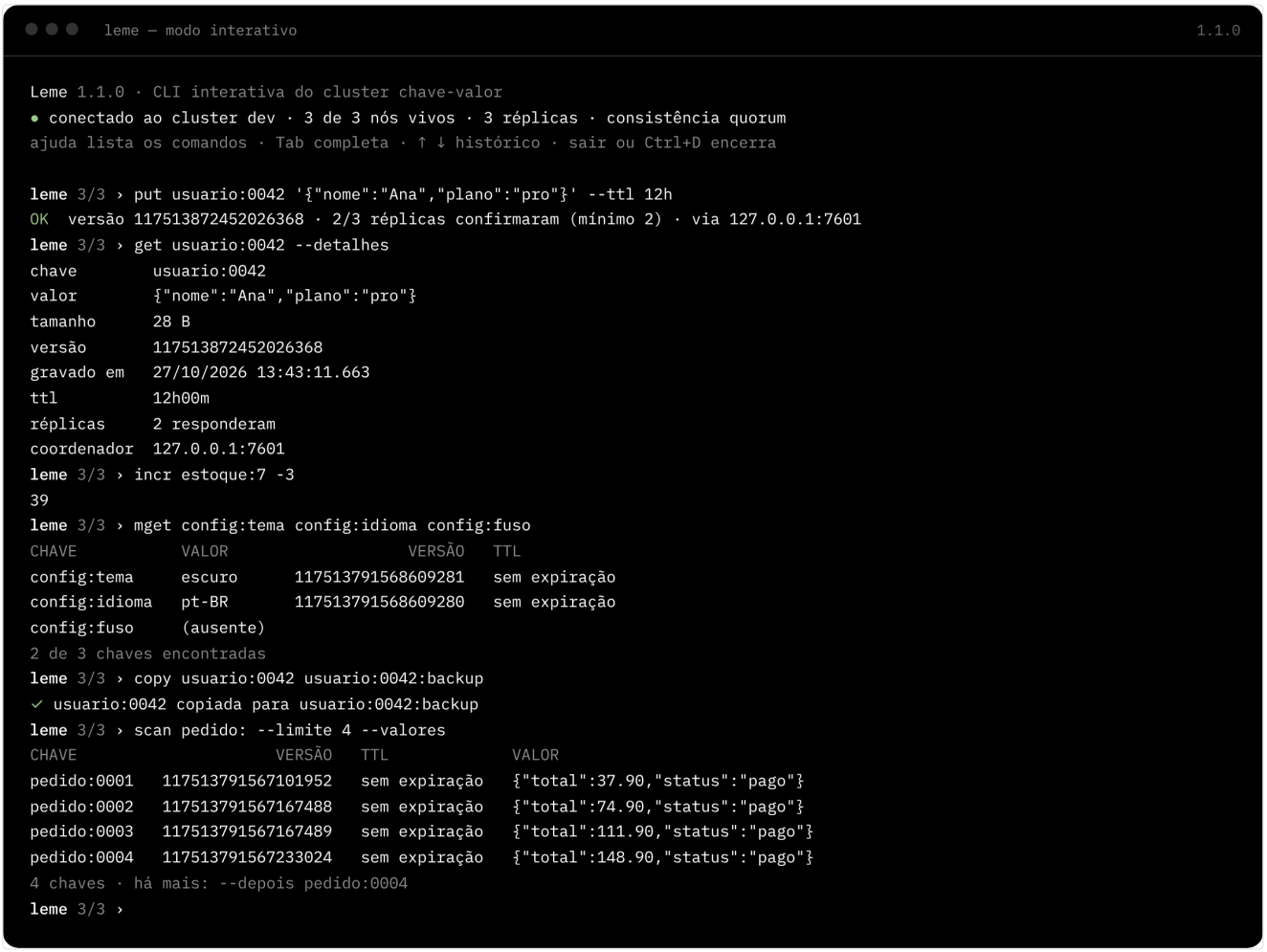
CAPÍTULO 05

Modo interativo

O modo interativo mantém uma conexão aberta com o cluster e oferece histórico, autocompletar e comandos de sessão, como num shell.

5.1 Abrindo

`leme` sem comando, ou `leme shell`, abre o modo interativo quando a entrada é um terminal. A conexão é aberta uma vez e reaproveitada por todos os comandos.



Modo interativo: put com TTL, get com detalhes, scan paginado, lista de nós e troca de consistência

O prompt `leme 3/3 ›` mostra quantos nós estão vivos do total conhecido. A sessão começa com a consistência e o formato do perfil.

Copyright (©) 2026 Erik Pimenta

5.2 Editando a linha

TECLA	O QUE FAZ
Tab	Completa comando, subcomando, opção, valor de opção ou chave já usada; com várias opções, mostra a lista
↑ ↓ ou **Ctrl+P Ctrl+N**	Navega no histórico
← → ou **Ctrl+B Ctrl+F**	Move o cursor
Ctrl+← Ctrl+→ ou **Alt+B Alt+F**	Pula palavras
Home End ou **Ctrl+A Ctrl+E**	Início e fim da linha
Ctrl+W	Apaga a palavra anterior
Ctrl+U·**Ctrl+K**	Apaga até o início·até o fim
Ctrl+L	Limpa a tela
Ctrl+C	Cancela a linha, ou o comando em andamento, sem sair
Ctrl+D	Sai, com a linha vazia

O histórico guarda até 1.000 comandos, sem repetir o anterior, e fica no arquivo `historico` da pasta de configuração.

5.3 Valores com espaços e caracteres especiais

A linha é dividida em palavras como num shell:

VOCÊ DIGITA	O VALOR GRAVADO
put nota:1 "texto com espaços"	`texto com espaços`
put perfil:1 '{"nome": "Ana"}'	`{"nome": "Ana"}` (aspas simples preservam tudo)
put k "aspas \"dentro\""	`aspas "dentro"`
put k linha\nnova	duas linhas
put saldo -5	`-5` (números negativos não viram opção)

5.4 Comandos de sessão

COMANDO	O QUE FAZ
ajuda ou **\?**	Lista os comandos por grupo
ajuda <comando>	Opções, apelidos e exemplos do comando
\saida tabela|json|texto	Muda o formato das próximas respostas
\consistencia um|quorum|todos	Muda a consistência das próximas operações
\tempo	Mostra o tempo de cada comando

Copyright (©) 2026 Erik Pimenta

COMANDO	O QUE FAZ
`\conectar host:porta[,host:porta]`	Troca de cluster sem sair
`\historico [n]`	Lista os últimos comandos
`\nos`	Atalho para `cluster nodes`
`limpar`	Limpa a tela
`sair`, `exit` ou `\q`	Encerra

Linhas que começam com `#` são comentários. Os comandos novos da 1.1.0 que funcionam numa só chamada, como `incr`, `copy`, `mget` e `doctor`, também estão disponíveis no modo interativo e no terminal web.

5.5 Confirmações

Comandos que mudam o cluster, como `node leave`, pedem confirmação no modo interativo. Responda `s` para seguir. Nos scripts, a confirmação é a opção `--confirmar`.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 06

Scripts e automação

Qualquer comando pode ser executado direto: a CLI conecta, executa, imprime e sai, com saídas e códigos de saída pensados para scripts.

6.1 Modo direto

```
leme put config:tema escuro
TEMA=$(leme get config:tema)
```

A saída de `get` é sempre o valor puro, sem enfeites, para caber em variáveis e pipes. Mensagens de erro vão para a saída de erro, nunca para a saída padrão.

6.2 Formatos de saída

FORMATO	QUANDO USAR	EXEMPLO
`tabela` (padrão)	Leitura humana; cores só quando a saída é um terminal	`leme cluster nodes`
`json`	Automação; objetos indentados, versões como texto para não perder precisão	`leme get k -o json | jq -r .valor`
`texto`	Pipes; uma linha por item, colunas separadas por tabulação	`leme keys pedido: -o texto`

6.3 Códigos de saída

CÓDIGO	SIGNIFICADO	EXEMPLO
0	Sucesso	
1	Erro inesperado	Disco cheio num nó
2	Uso incorreto	Faltou a chave, opção desconhecida
3	Chave não encontrada	`get` ou `exists` de chave ausente
4	Cluster indisponível	Nenhum nó respondeu, tempo esgotado
5	Quórum não atingido	`-c todos` com uma réplica fora
6	Conflito de versão	`--se-versao`, `--se-ausente` ou o destino de `copy` e `rename` já existe
7	Token recusado	
8	Diagnóstico encontrou falhas	`doctor` com pelo menos uma verificação em falha

Copyright (©) 2026 Erik Pimenta

```
if leme exists usuario:42; then echo "já existe"; fi

leme put estoque:7 41 --se-versao "$V"
case $? in
  0) echo "gravado" ;;
  6) echo "outra pessoa alterou antes, leia de novo" ;;
  *) echo "falhou" ;;
esac

leme doctor -o json > saude.json || echo "o cluster precisa de atenção"
```

6.4 Entrada padrão e arquivos

```
cat relatorio.json | leme put relatorio:2026-10 -
leme put foto:1 --arquivo foto.png
leme get foto:1 > copia.png
```

Valores binários são gravados e lidos byte a byte. Na tabela, aparecem como `<N bytes binários>`; no JSON, em `valor_base64`.

6.5 Arquivos de comandos

Um arquivo com um comando por linha roda com `leme shell --arquivo` ou por pipe:

```
# carga.leme
put usuario:1 '{"nome":"Ana"}'
put usuario:2 '{"nome":"Bruno"}'
incr contador:visitas
\saida json
count usuario:
```

```
leme shell --arquivo carga.leme
leme shell --arquivo carga.leme --parar-no-erro
printf 'put a 1\nget a\n' | leme
```

Sem `--parar-no-erro`, todas as linhas rodam e o código de saída é o da primeira falha. Com ele, o script para na primeira falha.

6.6 PowerShell

```
$env:LEME_NOS = "127.0.0.1:7401"
leme put sessao:1 ativa --ttl 15m
$usuario = leme get usuario:1 -o json | ConvertFrom-Json
if ($LASTEXITCODE -eq 3) { Write-Host "não existe" }
leme keys pedido: -o texto | ForEach-Object { leme del $_ }
leme export pedido: --arquivo pedidos.ndjson
```

Copyright (©) 2026 Erik Pimenta

6.7 Backup lógico e migrações

`export` e `import` levam chaves de um cluster para um arquivo e de volta, com TTL. Servem para backup lógico, para copiar dados entre ambientes e para carregar dados de teste:

```
leme --perfil producao export pedido: --arquivo pedidos.ndjson
leme --perfil homologacao import pedidos.ndjson --simular
leme --perfil homologacao import pedidos.ndjson --se-ausente
leme export config: --formato csv > config.csv
```

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 07

Referência de comandos

Todos os comandos aceitam as opções globais `--nos`, `--perfil`, `--token`, `--token-arquivo`, `--tls-ca`, `--tls-cert`, `--tls-chave`, `--tls-inseguro`, `--tempo-limite`, `-o` / `--saida`, `--sem-cor` e `--config`. Os comandos de dados aceitam `-c` / `--consistencia`.

7.1 get

`leme get <chave>` · apelidos `ler`, `obter`. Lê o valor mais recente entre as réplicas consultadas.

OPÇÃO	O QUE FAZ
`-d`, **`--detalhes`**	Mostra tamanho, versão, horário da gravação, TTL, réplicas que responderam e coordenador
`-c`, **`--consistencia`**	`um`, `quorum` ou `todos`

```
leme get usuario:42
leme get usuario:42 --detalhes -c todos
leme get config:tema -o json
```

7.2 put

`leme put <chave> <valor | ->` · apelidos `set`, `gravar`. Grava um valor.

OPÇÃO	O QUE FAZ
`-t`, **`--ttl`**	Expira depois da duração: `90` (segundos), `30s`, `15m`, `2h`
`--se-versao <v>`	Só grava se a versão atual for `v` (código 6 se não for)
`-n`, **`--se-ausente`**	Só grava se a chave não existir
`-f`, **`--arquivo`**	Lê o valor de um arquivo
`-c`, **`--consistencia`**	Réplicas que precisam confirmar

A resposta mostra a versão gravada, quantas réplicas confirmaram até a resposta, o mínimo exigido e o nó coordenador. Limites: chave até 64 KiB, valor até 32 MiB.

7.3 del

`leme del <chave>...` · apelidos `delete`, `apagar`, `rm`. Apaga uma ou várias chaves. `--se-versao` funciona com uma chave por vez. Apagar uma chave que não existe também grava a lápide e termina com sucesso.

Copyright (©) 2026 Erik Pimenta

7.4 exists, ttl e expire

COMANDO	O QUE FAZ	SAÍDA EM TEXTO
`exists <chave>`	Código 0 se existe, 3 se não	`1` ou `0`
`ttl <chave>`	Tempo até expirar	milissegundos, ou `-1` sem expiração
`expire <chave> <duração>`	Novo TTL a partir de agora; `0` remove a expiração	`OK`

7.5 scan, keys e count

`leme scan [prefixo]` · apelidos `varrer`, `listar`. Lista as chaves em ordem lexicográfica, consultando todos os nós vivos.

OPÇÃO	O QUE FAZ
`-l`, `--limite`	Chaves por página, de 1 a 10.000 (padrão 50)
`--depois <chave>`	Começa depois desta chave: a próxima página
`-v`, `--valores`	Inclui os valores
`-a`, `--todas`	Percorre todas as páginas

O rodapé indica se há mais páginas e a chave para continuar. `keys [prefixo]` lista só os nomes (até `--limite`, padrão 1.000). `count [prefixo]` conta percorrendo as páginas.

7.6 incr e decr

`leme incr <chave> [delta]` e `leme decr <chave> [delta]` · apelidos `incrementar`, `decrementar`. Somam ou subtraem `delta` (padrão 1) de um contador guardado como número inteiro em texto e imprimem o valor novo. Uma chave ausente começa em 0.

OPÇÃO	O QUE FAZ
`-t`, `--ttl`	TTL aplicado quando o contador é criado
`-c`, `--consistencia`	Consistência da leitura e da escrita

O contador é atômico do ponto de vista da CLI: ela lê a versão atual, grava com `--se-versao` e, em caso de conflito de versão, lê de novo e repete, com espera crescente. Um valor que não é número inteiro termina com o código 2.

```
leme incr pagina:inicio:visitas
leme incr estoque:sku-1042 -5
leme decr vagas:turma-3
leme incr limite:api:203.0.113.7 --ttl 1m
```

7.7 copy e rename

`leme copy <origem> <destino>` · apelidos `copiar`, `cp`. `leme rename <origem> <destino>` · apelidos `renomear`, `mv`. Copiam o valor e o TTL restante da origem para o destino; `rename` depois apaga a origem, condicionado à versão lida, para não apagar uma escrita feita no meio do caminho.

Copyright (©) 2026 Erik Pimenta

OPÇÃO	O QUE FAZ
`-s`, **`--substituir`**	Sobrescreve o destino; sem ela, um destino existente termina com o código 6
`-c`, **`--consistencia`**	Consistência das leituras e escritas

```
leme copy config:tema config:tema:backup
leme rename sessao:temp sessao:9f3c2a
leme rename usuario:42 usuario:000042 --substituir
```

7.8 mget e mset

`leme mget <chave>...` · apelido `ler-varios`. Lê várias chaves em paralelo e mostra chave, valor e versão; chaves ausentes aparecem marcadas, sem interromper as outras. `leme mset <chave=valor>...` · apelido `gravar-varios`. Grava vários pares em paralelo; `--ttl` vale para todos.

```
leme mget usuario:1 usuario:2 usuario:3
leme mset config:moeda=BRL config:idioma=pt-BR config:fuso=America/Sao_Paulo
leme mset sessao:a=1 sessao:b=2 --ttl 30m
```

7.9 export e import

`leme export [prefixo]` · apelido `exportar`. Exporta as chaves do prefixo (ou todas) com o valor, o TTL restante e a versão. Valores de texto vão como texto; binários, em base64. Sem `--arquivo`, escreve na saída padrão, e o resumo vai para a saída de erro.

OPÇÃO	O QUE FAZ
`-a`, **`--arquivo <caminho>`**	Grava no arquivo em vez da saída padrão
`--formato ndjson|json|csv`	NDJSON (padrão, um objeto por linha), um único JSON ou CSV; com `--arquivo`, deduzido pela extensão
`-c`, **`--consistencia`**	Consistência das leituras

```
{"chave":"usuario:42","valor":"{\"nome\":\"Ana\"}","codificacao":"texto","ttl_ms":1799000,"versao":
1187556298427269l2}
```

`leme import <arquivo | ->` · apelido `importar`. Lê um arquivo exportado (ou a entrada padrão) e grava as chaves, preservando o TTL. O formato é detectado pela extensão do arquivo ou pelo conteúdo; na dúvida, NDJSON.

OPÇÃO	O QUE FAZ
`--formato ndjson|json|csv`	Força o formato
`-n`, **`--se-ausente`**	Não sobrescreve chaves que já existem (contam como ignoradas)
`--prefixo <texto>`	Acrescenta um prefixo a cada chave importada
`--simular`	Só lê, valida e conta; não grava nada

Copyright (©) 2026 Erik Pimenta

O relatório mostra lidos, gravados, ignorados e falhas. Cada chave importada recebe uma versão nova do cluster de destino: o campo `versao` do arquivo é informativo.

7.10 locate e ping

`locate <chave>` (apelido `localizar`) mostra a réplica primária e as demais, com endereço e estado. `ping [nó]` mede a latência até cada nó, ou até um só, e termina com código 4 se nenhum responder.

7.11 cluster status, nodes e ring

COMANDO	MOSTRA
`cluster status`	Saúde (saudável, degradado ou sem quórum), nós por estado, replicação, chaves aproximadas, disco, operações, latência p99 de leitura e escrita, dicas e reparos
`cluster nodes` (`nos`, `membros`)	Cada membro com estado, endereço, tempo no ar, latência medida pelo gossip, fatia do anel e versão
`cluster ring` (`anel`)	A fatia do espaço de chaves de cada nó, com barra proporcional

7.12 node stats, compact e leave

COMANDO	O QUE FAZ
`node stats <nó>`	Versão, tempo no ar, TLS e token, conexões, memória, chaves, lápides, disco, fragmentação, fsync, compactações, dicas, reparos e a latência de cada operação (média, p50, p95 e p99)
`node compact <nó | todos>`	Compacta os segmentos e mostra o tamanho antes e depois, os registros removidos e o tempo
`node leave <nó> --confirmar`	Retira o nó do cluster: ele anuncia a saída, o anel é recalculado e as chaves vão para as novas réplicas

O `<nó>` pode ser o identificador (`no-2`) ou o endereço (`10.0.0.12:7400`).

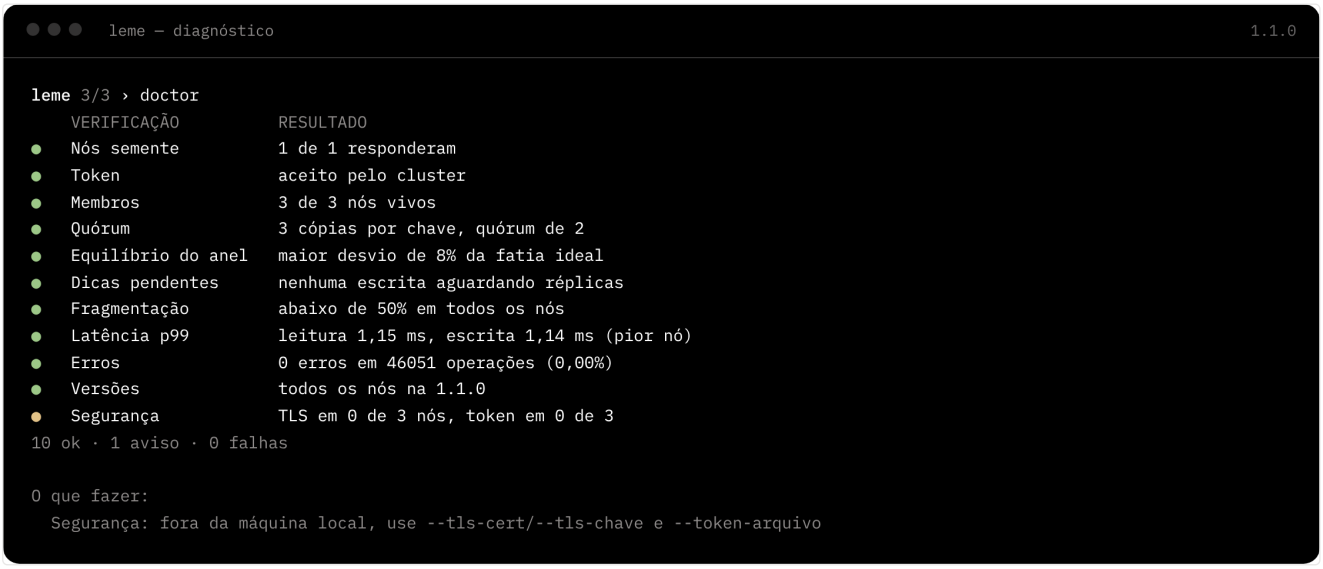
7.13 doctor

`leme doctor` · apelido `diagnostico`. Roda uma bateria de verificações e mostra, para cada uma, o estado (`ok`, `aviso` ou `falha`), o que foi medido e uma sugestão quando há algo a fazer.

VERIFICAÇÃO	O QUE CONFERE
Nós semente	Os nós informados em `--nos` ou no perfil respondem
Token	O cluster aceita o token da sessão
Membros	Quantos membros estão vivos, suspeitos ou fora do ar
Quórum e replicação	Se há nós vivos suficientes para o quórum e para o número de cópias configurado
Equilíbrio do anel	Quanto a fatia de cada nó se afasta da divisão perfeita
Estatísticas	Se todos os nós do anel respondem às estatísticas
Dicas pendentes	Escritas aguardando réplicas fora do ar

Copyright (©) 2026 Erik Pimenta

VERIFICAÇÃO	O QUE CONFERE
Fragmentação	Nós com mais de 50% do disco ocupado por versões antigas e lápides
Latência p99	p99 de leitura e escrita de cada nó
Erros	Proporção de operações com erro
Versões	Se todos os nós rodam a mesma versão do `lemed`
Segurança	Se TLS e token estão ligados em todos os nós



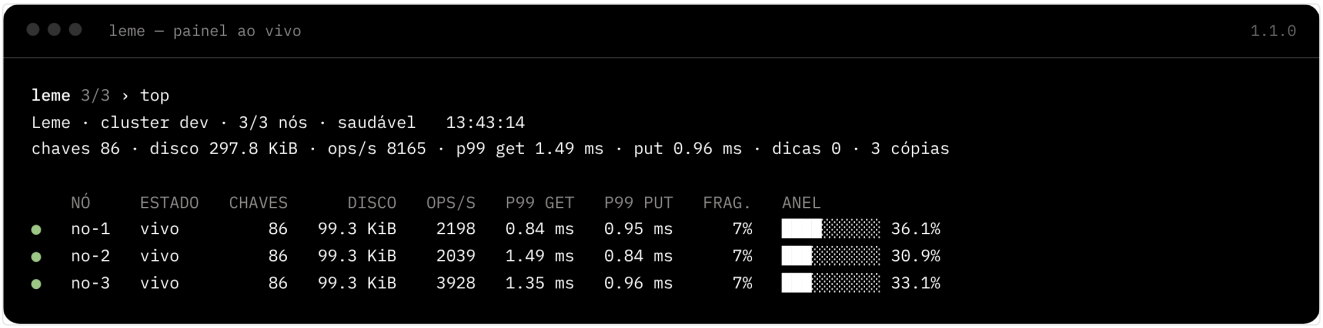
leme doctor no terminal: verificações com ok, aviso e falha e as sugestões

Termina com o código 0 quando tudo está ok ou só há avisos, e com o código 8 quando alguma verificação falha. Com `-o json`, a lista sai pronta para monitoramento.

7.14 watch e top

`leme watch <chave>` ou `leme watch --prefixo <p>` · apelido `vigiar`. Acompanha uma chave ou um prefixo e imprime cada mudança, com horário: criada, alterada (com a versão nova), apagada ou expirada. Roda até `Ctrl+C`; `-i` / `--intervalo` define a frequência de consulta (padrão `1s`) e `--vezes` para depois de N consultas, útil em scripts.

`leme top` · apelido `painel`. Painel ao vivo no terminal, no estilo do `top`: estado do cluster, e para cada nó estado, operações por segundo, p99 de leitura e escrita, chaves e disco. Atualiza a cada `-i` / `--intervalo` (padrão `2s`) e sai com `Ctrl+C` ou, com `--vezes`, depois de N atualizações.



leme top: painel ao vivo com os nós, operações por segundo, p99, chaves e disco

Copyright (©) 2026 Erik Pimenta

```
leme watch config:tema
leme watch --prefixo pedido: --intervalo 500ms
leme top --intervalo 1s
```

7.15 Ferramentas

COMANDO	O QUE FAZ
shell	Modo interativo; `--arquivo` e `--parar-no-erro` para scripts
console	Console web e API REST; `--endereco` e `--token-console`
dev	Cluster local: `--quantidade`, `--porta`, `--replicas`, `--dados`, `--console`, `--logs`, `--limpar`
bench	Carga sintética: `--operacoes`, `--concorrencia`, `--tamanho`, `--chaves`, `--leituras`, `--prefixo`
config	Perfis de conexão
version	Versão, Go, sistema e commit
completion	Autocompletar do shell

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 08

Console web

O console mostra o cluster no navegador, com a mesma conexão da CLI: visão geral, nós, anel, chaves, terminal, diagnóstico e integrações, em tema claro ou escuro.

8.1 Abrindo

```
leme console                                          # http://127.0.0.1:8080/
leme --perfil producao console --endereco 127.0.0.1:9090
leme console --endereco 0.0.0.0:8080 --token-console "$(openssl rand -hex 16)"
```

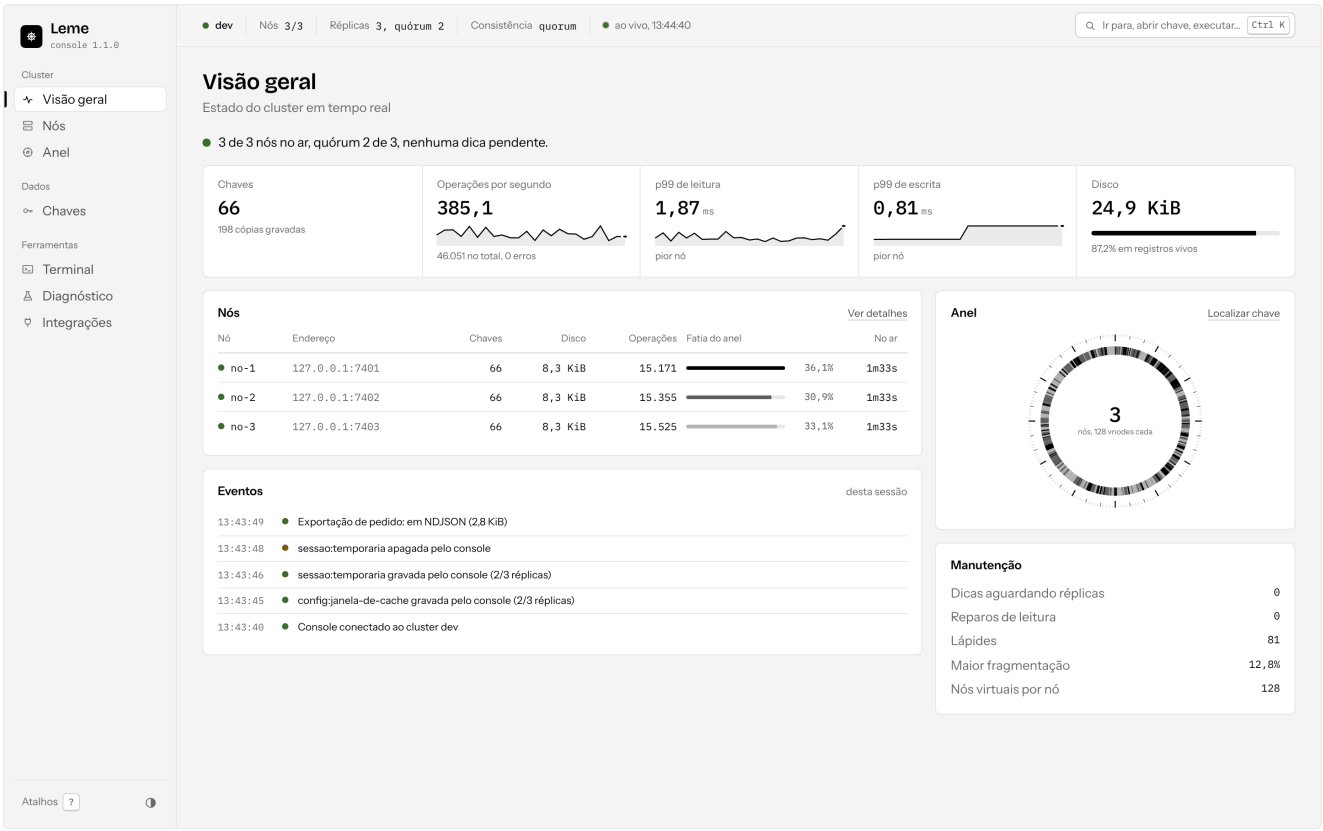
O console usa a mesma conexão da CLI. Por segurança, escuta só em `127.0.0.1`; para expor na rede é obrigatório definir `--token-console` (ou `LEME_TOKEN_CONSOLE`). O endereço mostrado no terminal já leva o token; o navegador troca o token por um cookie e remove-o da barra de endereço.

8.2 Organização da tela

A barra lateral agrupa as páginas em três blocos: **Cluster** (Visão geral, Nós, Anel), **Dados** (Chaves) e **Ferramentas** (Terminal, Diagnóstico, Integrações). A página aberta fica marcada. No topo, uma faixa de status mostra sempre o nome do cluster, quantos nós estão vivos, réplicas e quórum, a consistência da sessão, o indicador de atualização ao vivo e o atalho da busca.

Copyright (©) 2026 Erik Pimenta

8.3 Visão geral



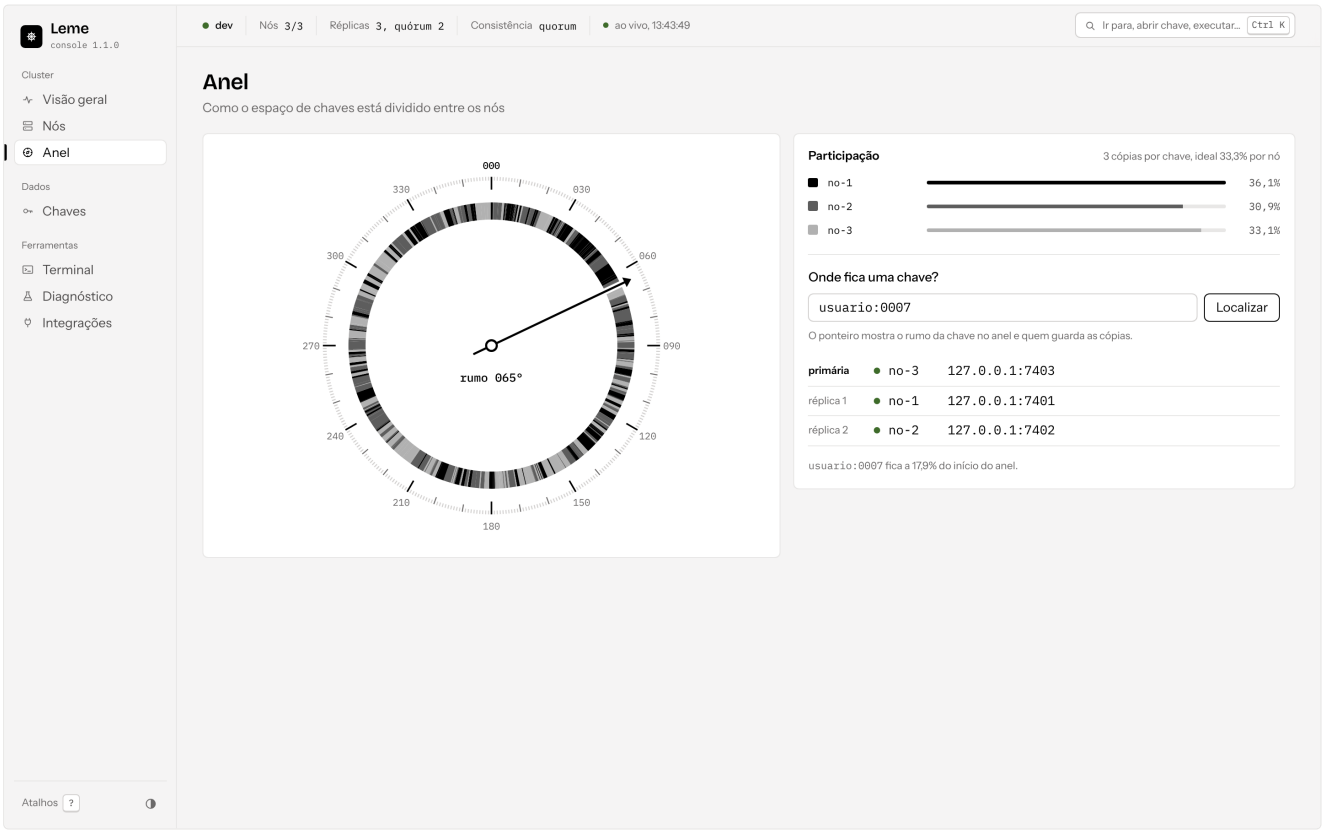
Visão geral: frase de estado, métricas com sparklines, nós, anel e eventos

Uma frase resume o estado do cluster, como “3 de 3 nós no ar · quórum 2 de 3 · nenhuma dica pendente”. Abaixo ficam as métricas principais (chaves, operações por segundo, latência p99 de leitura e de escrita do pior nó, disco e pendências), cada uma com uma sparkline dos últimos minutos, a tabela dos nós com a participação de cada um no anel, o anel desenhado como um mostrador e o feed de eventos: mudanças de estado dos nós e gravações e remoções feitas pelo console. A página se atualiza sozinha a cada poucos segundos.

8.4 Nós e anel

A página Nós mostra, para cada membro, estado, endereço, chaves, lápides, disco, fragmentação, operações, p99 de leitura, fatia do anel e tempo no ar. A página Anel desenha as faixas de cada nó num mostrador de 360 graus e responde onde fica uma chave: digite a chave, clique em Localizar e veja a posição no anel, a primária e as réplicas.

Copyright (©) 2026 Erik Pimenta



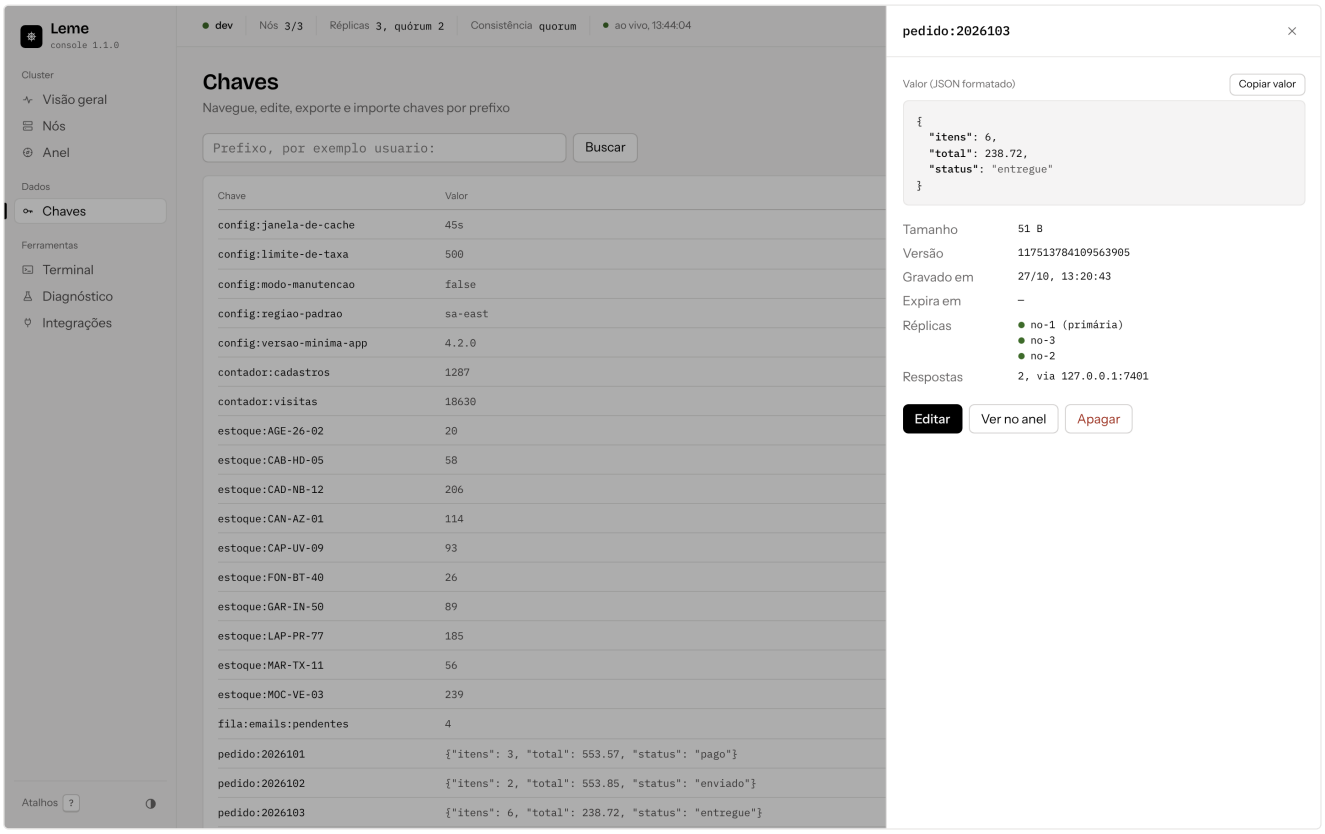
Anel com a participação de cada nó e a localização de uma chave

8.5 Chaves

Busque por prefixo e carregue mais páginas com Carregar mais. Clique numa linha para abrir a gaveta com o valor (JSON aparece formatado), tamanho, versão, horário, TTL, réplicas e o nó que respondeu, com um botão para copiar o valor. Na gaveta, Editar grava com a opção de só gravar se ninguém alterou desde a versão aberta, e Apagar pede confirmação. Nova chave abre um formulário com chave, valor e TTL.

Exportar baixa as chaves do prefixo atual em NDJSON, JSON ou CSV, no mesmo formato do `leme export`. **Importar** envia um arquivo exportado; a opção de simular mostra o relatório sem gravar, e a de só gravar ausentes preserva o que já existe.

Copyright (©) 2026 Erik Pimenta



Gaveta de uma chave com o valor formatado, versão, TTL e réplicas

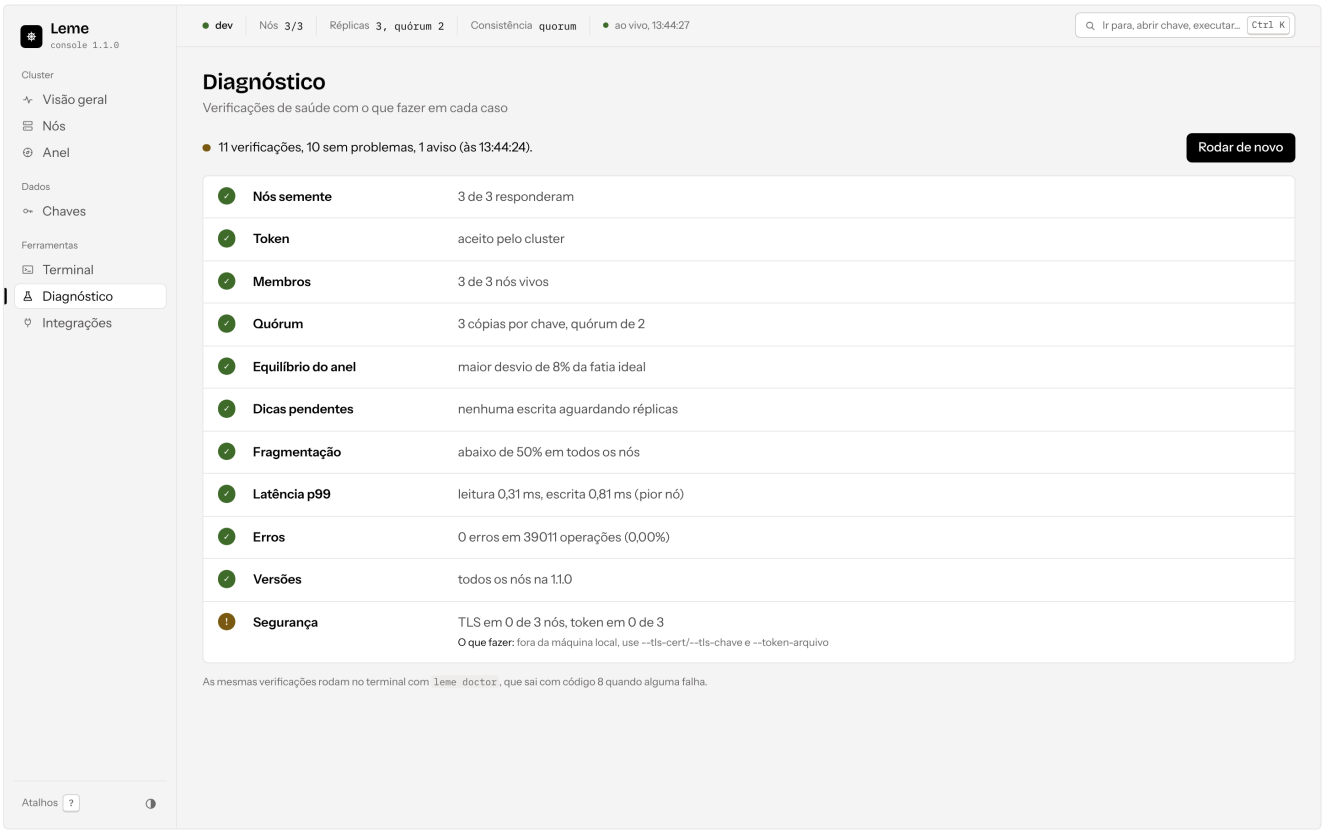
8.6 Terminal

O terminal web executa os mesmos comandos da CLI, incluindo `incr`, `copy`, `mget` e `doctor`, com histórico em `↑` e `↓` e `Ctrl+L` para limpar. Comandos perigosos exigem `--confirmar`.

8.7 Diagnóstico

A página Diagnóstico roda as mesmas verificações do `leme doctor` e mostra cada uma com o estado, o que foi medido e a sugestão. **Rodar de novo** repete a bateria, por exemplo depois de compactar um nó ou de subir um nó que estava fora.

Copyright (©) 2026 Erik Pimenta

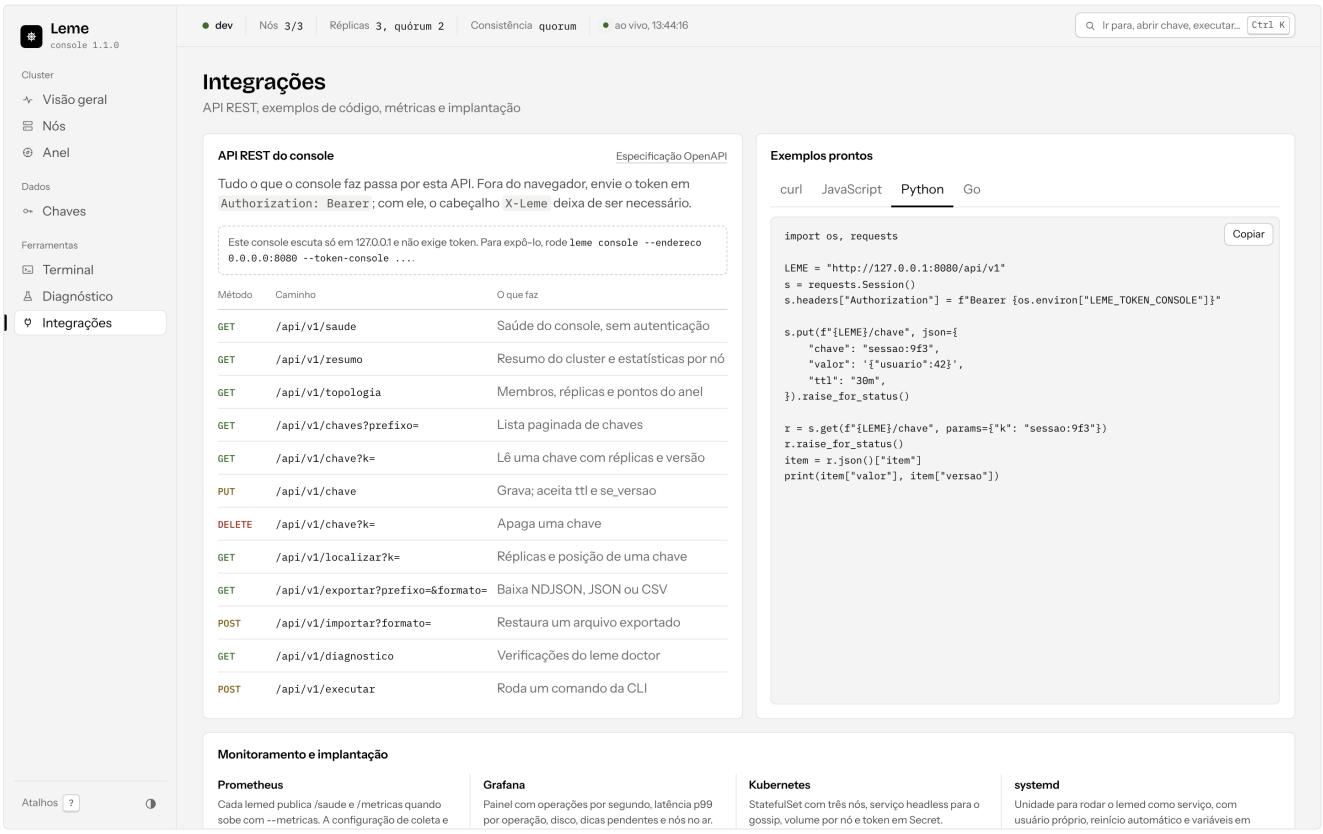


Diagnóstico com as verificações do cluster

8.8 Integrações

A página Integrações reúne o que é preciso para ligar o Leme a outros sistemas: as rotas da API REST, trechos prontos em curl, JavaScript, Python e Go já preenchidos com o endereço do console e o token (mascarado, com botões para mostrar e copiar), o link para a especificação OpenAPI e os arquivos de Prometheus, Grafana, Kubernetes e systemd do capítulo 9.

Copyright (©) 2026 Erik Pimenta



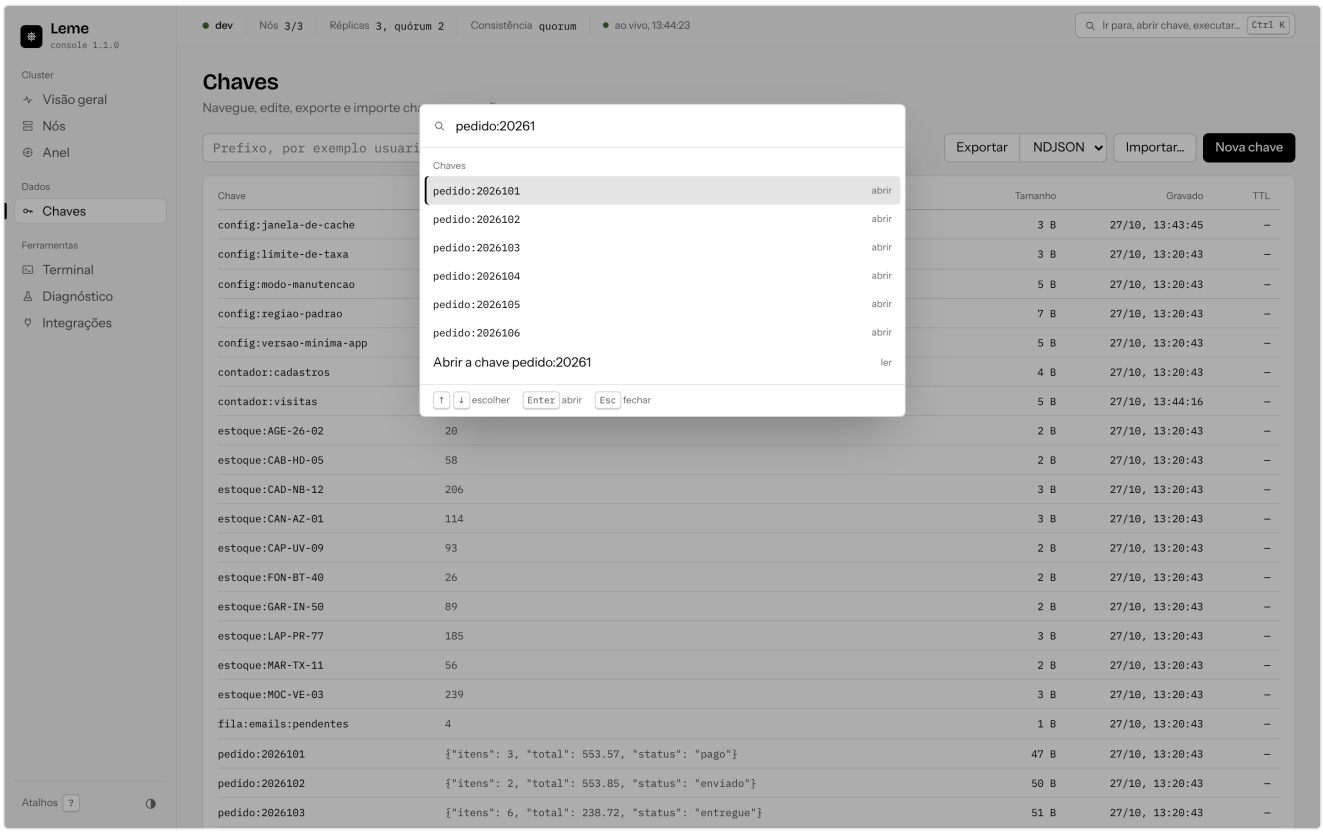
Integrações com os trechos de código prontos

8.9 Paleta de comandos e atalhos

`Ctrl+K` (`Cmd+K` no macOS) abre a paleta de comandos: digite para ir a uma página, abrir uma chave pelo nome, executar um comando no terminal ou alternar o tema.

ATALHO	O QUE FAZ
`Ctrl+K` ou **`Cmd+K`**	Abre a paleta de comandos
`/`	Foca a busca da página
`g` e depois **`o`**, **`n`**, **`a`**, **`c`**, **`t`**, **`d`** ou **`i`**	Vai para Visão geral, Nós, Anel, Chaves, Terminal, Diagnóstico ou Integrações
`?`	Mostra a lista de atalhos
`Esc`	Fecha a paleta, a gaveta ou o formulário

Copyright (©) 2026 Erik Pimenta



Paleta de comandos aberta sobre a visão geral

8.10 Tema, celular e acessibilidade

O botão de tema alterna entre claro e escuro, e a escolha fica salva no navegador. No celular, o menu vira uma gaveta e os blocos ficam em uma coluna. Os contrastes seguem o nível AA, o foco do teclado fica sempre visível, os avisos são anunciados aos leitores de tela e as animações respeitam a preferência por menos movimento.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 09

API REST e integrações

O console expõe uma API HTTP com JSON, a mesma que as páginas usam, e o projeto traz arquivos prontos para Kubernetes, systemd, Prometheus e Grafana.

9.1 Autenticação

A API vive em `/api/v1`, no mesmo endereço do console. Sem `--token-console`, só aceita conexões de `127.0.0.1`. Com token, cada pedido se autentica de uma destas formas:

FORMA	QUANDO USAR
`Authorization: Bearer <token-console>`	Scripts e outros sistemas. Dispensa o cabeçalho `X-Leme`, porque o navegador nunca envia esse cabeçalho sozinho
Cookie **`leme_console`**	O próprio console, depois de abrir o endereço com `?token=`
`?token=<token-console>`	Só para abrir o console pela primeira vez

Pedidos que alteram dados sem Bearer precisam do cabeçalho `X-Leme: 1`. O token é comparado em tempo constante.

9.2 Rotas

MÉTODO E ROTA	O QUE FAZ
`GET /api/v1/saude`	Estado e versão do console; não exige token
`GET /api/v1/openapi.json`	Especificação OpenAPI 3.1 de todas as rotas; não exige token
`GET /api/v1/resumo`	O mesmo resumo do `cluster status`
`GET /api/v1/topologia`	Membros e anel
`GET /api/v1/chaves?prefixo=&limite=&depois=`	Página de chaves (até 500 por página)
`GET /api/v1/chave?k=<chave>`	Valor e metadados de uma chave
`PUT /api/v1/chave`	Grava; corpo JSON com `chave`, `valor`, `ttl` e `se_versao`
`DELETE /api/v1/chave?k=<chave>`	Apaga
`GET /api/v1/exportar?prefixo=&formato=`	Exportação em NDJSON, JSON ou CSV, como anexo
`POST /api/v1/importar?formato=&se_ausente=&simular=&prefixo=`	Importa o arquivo enviado no corpo e devolve o relatório
`GET /api/v1/diagnostico`	As verificações do `doctor`, com a contagem de ok, avisos e falhas
`GET /api/v1/localizar?k=<chave>`	Posição da chave no anel e réplicas

Copyright (©) 2026 Erik Pimenta

MÉTODO E ROTA	O QUE FAZ
`POST /api/v1/executar`	Executa um comando da CLI e devolve a saída
`GET /api/v1/comandos`	Lista os comandos disponíveis
`GET /api/v1/integracao`	Endereço, versão e proteção do console, usados pela página Integrações

```
export LEME_API=http://127.0.0.1:8080/api/v1
export LEME_TOKEN_CONSOLE=...
AUT="Authorization: Bearer $LEME_TOKEN_CONSOLE"
curl -s -H "$AUT" "$LEME_API/chave?k=usuario:42"
curl -s -X PUT -H "$AUT" -H "Content-Type: application/json" \
     --data '{"chave":"sessao:9f3c2a","valor":"ativa","ttl":"30m"}' "$LEME_API/chave"
curl -s -H "$AUT" "$LEME_API/exportar?prefixo=pedido:&formato=ndjson" -o pedidos.ndjson
curl -s -X POST -H "$AUT" --data-binary @pedidos.ndjson "$LEME_API/importar?formato=ndjson&simular=1"
curl -s -H "$AUT" "$LEME_API/diagnostico" | jq '.verificacoes[] | select(.estado != "ok")'
```

A página Integrações do console mostra os mesmos exemplos em JavaScript, Python e Go, já com o endereço certo. A especificação em `openapi.json` gera clientes em outras linguagens com qualquer gerador de OpenAPI.

9.3 Kubernetes

`deploy/kubernetes/leme.yaml` sobe três nós como um StatefulSet, com um Service headless para os nós se encontrarem pelo nome (`leme-0.leme`, `leme-1.leme`…), um volume persistente por nó, um PodDisruptionBudget que deixa cair só um nó por vez e o console num Deployment com o seu Service. Os tokens do cluster e do console ficam no Secret `leme-token`.

```
docker build -t leme:1.1.0 .
kubectl create namespace leme
kubectl -n leme create secret generic leme-token \
    --from-literal=token="$(openssl rand -hex 24)" \
    --from-literal=token-console="$(openssl rand -hex 24)"
kubectl -n leme apply -f deploy/kubernetes/leme.yaml
kubectl -n leme rollout status statefulset/leme
kubectl -n leme port-forward svc/leme-console 8080:8080
kubectl -n leme exec -it leme-0 -- leme --nos leme-0.leme:7400 doctor
```

Para crescer, aumente as réplicas do StatefulSet: cada nó novo entra pelos anteriores e recebe as chaves que passam a ser dele. Para reduzir, retire o nó com `node leave` antes de diminuir as réplicas.

9.4 systemd

`deploy/systemd/lemed.service` é a unidade do capítulo 10, com usuário próprio, reinício automático e limites de arquivos, e lê as variáveis de `/etc/leme/lemed.env` (modelo em `deploy/systemd/lemed.env`).

Copyright (©) 2026 Erik Pimenta

```
sudo useradd --system --home /var/lib/leme --shell /usr/sbin/nologin leme
sudo install -m 0755 bin/lemed /usr/local/bin/lemed
sudo install -d -o leme -g leme -m 0750 /var/lib/leme /etc/leme
sudo install -m 0640 -o root -g leme deploy/systemd/lemed.env /etc/leme/lemed.env
sudo install -m 0644 deploy/systemd/lemed.service /etc/systemd/system/lemed.service
sudo systemctl daemon-reload && sudo systemctl enable --now lemed
journalctl -u lemed -f
```

9.5 Prometheus e Grafana

`deploy/prometheus/prometheus.yml` coleta `/metricas` de cada nó e carrega `deploy/prometheus/alertas.yml`. `deploy/grafana/leme-dashboard.json` é um painel pronto para importar no Grafana, com operações por segundo, latência por operação, chaves, disco e fragmentação, dicas e reparos; `deploy/grafana/provisionamento` já registra a fonte de dados e o painel.

No Docker Compose, o perfil `observabilidade` sobe os dois junto com o cluster:

```
docker compose --profile observabilidade up -d
# Prometheus em http://127.0.0.1:9090 · Grafana em http://127.0.0.1:3000
```

ALERTA	QUANDO DISPARA
LemeNoForaDoAr	O alvo de um nó não responde por mais de 1 minuto
LemeDicasAcumulando	Mais de 1.000 dicas pendentes no cluster por 5 minutos
LemeLatenciaAlta	p99 de GET, PUT ou DEL acima de 250 ms por 10 minutos
LemeTaxaDeErros	Mais de 5% das operações com erro por 10 minutos
LemeFragmentacao	Fragmentação acima de 60% num nó com mais de 1 GB em disco, por 1 hora

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 10

Operando o cluster

Este capítulo mostra como subir, manter, crescer e monitorar um cluster de produção.

10.1 O nó lemed

FLAG	VARIÁVEL	PADRÃO	O QUE FAZ
`--id`	`LEME_ID`	`host-porta`	Identificador único e estável do nó
`--cluster`	`LEME_CLUSTER`	`leme`	Nome do cluster
`--endereco`	`LEME_ENDERECO`	`127.0.0.1:7400`	Onde escuta
`--anunciar`	`LEME_ANUNCIAR`	o endereço	Endereço que os outros usam para chegar até ele
`--entrar`	`LEME_ENTRAR`	vazio	Nós existentes para entrar no cluster
`--dados`	`LEME_DADOS`	`dados`	Pasta dos dados; cada nó usa a subpasta com o seu id
`--replicas`	`LEME_REPLICAS`	3	Cópias de cada chave
`--vnodes`	`LEME_VNODES`	128	Nós virtuais por nó
`--token`, `--token-arquivo`	`LEME_TOKEN`, `LEME_TOKEN_ARQUIVO`	vazio	Token de clientes e pares
`--tls-cert`, `--tls-chave`, `--tls-ca`	`LEME_TLS_*`	vazio	TLS e mTLS
`--metricas`	`LEME_METRICAS`	vazio	HTTP de `/saude` e `/metricas`
`--sincronizar`	`LEME_SINCRONIZAR`	`intervalo`	`sempre`, `intervalo` ou `nunca`
`--segmento-mb`	`LEME_SEGMENTO_MB`	64	Tamanho de cada segmento de log
`--graca-lapide`	—	24h	Tempo mínimo de uma lápide no disco
`--gossip`	—	1s	Intervalo do gossip
`--tempo-operacao`	—	3s	Limite de uma operação entre nós
`--max-conexoes`	`LEME_MAX_CONEXOES`	4096	Conexões simultâneas
`--log`, `--log-formato`	`LEME_LOG`, `LEME_LOG_FORMATO`	`info`, `texto`	Logs; `json` para coletores

> **Mesmos valores em todos os nós**
>
> `--cluster`, `--replicas`, `--vnodes` e o token precisam ser iguais em todos os nós. Nós de clusters com nomes diferentes recusam a conversa.

Copyright (©) 2026 Erik Pimenta

10.2 Subindo um cluster

```
lemed --id no-1 --endereco 0.0.0.0:7400 --anunciar 10.0.0.11:7400 --dados /var/lib/leme \
      --token-arquivo /etc/leme/token --metricas 0.0.0.0:9400
lemed --id no-2 --endereco 0.0.0.0:7400 --anunciar 10.0.0.12:7400 --dados /var/lib/leme \
      --token-arquivo /etc/leme/token --metricas 0.0.0.0:9400 --entrar 10.0.0.11:7400
lemed --id no-3 ... --entrar 10.0.0.11:7400,10.0.0.12:7400
```

Use pelo menos três nós para tolerar a queda de um com quórum. Confira com `leme --nos 10.0.0.11:7400 cluster nodes` ou, de uma vez, com `leme --nos 10.0.0.11:7400 doctor`.

10.3 Como serviço do systemd

A unidade abaixo está pronta em `deploy/systemd/lemed.service`; a versão do arquivo lê as variáveis de `/etc/leme/lemed.env` (capítulo 9).

```
[Unit]
Description=Leme - no do cluster
After=network-online.target
Wants=network-online.target

[Service]
User=leme
Environment=LEME_ID=no-1
Environment=LEME_ENDERECO=0.0.0.0:7400
Environment=LEME_ANUNCIAR=10.0.0.11:7400
Environment=LEME_DADOS=/var/lib/leme
Environment=LEME_ENTRAR=10.0.0.12:7400,10.0.0.13:7400
Environment=LEME_TOKEN_ARQUIVO=/etc/leme/token
Environment=LEME_METRICAS=0.0.0.0:9400
ExecStart=/usr/local/bin/lemed
Restart=on-failure
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
```

Ao receber `SIGTERM`, o nó para de aceitar conexões, espera as operações em andamento por até 15 segundos, sincroniza o log e sai.

10.4 Crescendo o cluster

Suba o novo nó com `--entrar` apontando para os existentes. Em segundos ele aparece em `cluster nodes`, o anel é recalculado e cada nó envia ao novo as chaves que passaram a ser dele. Com nós virtuais, um quarto nó recebe perto de 25% das chaves, tiradas um pouco de cada um dos outros.

10.5 Retirando um nó

```
leme node leave no-3 --confirmar
leme cluster nodes                  # no-3 aparece como saiu
```

Copyright (©) 2026 Erik Pimenta

O nó transfere todas as suas chaves para as novas réplicas antes de sair do anel. Depois disso o processo pode ser desligado. Para devolver um nó retirado, apague a pasta de dados dele e suba de novo com `--entrar`.

10.6 Falhas e recuperação

SITUAÇÃO	O QUE ACONTECE	O QUE FAZER
Um nó cai	Vira suspeito em 3s e morto em 10s; com quórum, leituras e escritas seguem; escritas para ele viram dicas	Nada; ao voltar, recebe as dicas e o reparo de leitura completa o resto
Dois de três nós caem	Operações com `quorum` falham com código 5; `-c um` ainda funciona	Suba os nós; `cluster status` mostra "sem quórum" e `doctor` falha na verificação de quórum
Queda de energia	Na volta, o nó relê o log, valida o CRC de cada registro e trunca uma escrita incompleta no fim	Nada; veja "registros descartados" em `node stats`
Disco perdido num nó	O nó volta vazio	Retire-o com `node leave` e suba-o de novo, vazio, com outro id e `--entrar`: os outros nós enviam a ele as chaves que passam a ser dele

10.7 Compactação

As escritas só acrescentam registros, então versões antigas, lápides e chaves expiradas ocupam espaço até a compactação. Rode quando a fragmentação de `node stats` passar de uns 50% (o `doctor` avisa):

```
leme node compact todos
```

A compactação reescreve os segmentos fechados sem bloquear leituras e escritas e só remove lápides com mais de `--graca-lapide` (24h).

10.8 Backup

Há duas formas, que se completam:

FORMA	COMO	QUANDO USAR
Física	Rode `node compact` (que fecha o segmento ativo), copie a pasta `dados/<id>` e guarde	Restaurar um nó inteiro, rápido, com o histórico de versões
Lógica	`leme export --arquivo backup.ndjson` e, para restaurar, `leme import backup.ndjson`	Restaurar algumas chaves, levar dados para outro cluster, guardar um arquivo legível

Como cada chave tem três réplicas, um backup físico por nó em horários diferentes já protege contra a perda simultânea de todos. O backup lógico preserva o TTL restante e os valores binários; as chaves restauradas recebem versões novas.

Copyright (©) 2026 Erik Pimenta

10.9 Monitoramento

Com `--metricas`, cada nó expõe `GET /saude` (200 com `{"status":"ok"}`, 503 durante a saída), `GET /v1/estatisticas`, `GET /v1/topologia` e `GET /metricas`:

MÉTRICA	TIPO	O QUE MEDE
`leme_operacoes_total{op}`	contador	Operações atendidas por tipo
`leme_erros_total{op}`	contador	Operações com erro
`leme_operacao_segundos{op}`	histograma	Latência
`leme_chaves`, `leme_lapides`	medidor	Registros no nó
`leme_bytes_disco`, `leme_bytes_vivos`, `leme_segmentos`	medidor	Disco e fragmentação
`leme_dicas_pendentes`	medidor	Escritas esperando réplicas fora do ar
`leme_reparos_leitura_total`, `leme_compactacoes_total`	contador	Reparos e compactações
`leme_conexoes`, `leme_memoria_bytes`, `leme_tempo_no_ar_segundos`	medidor	Processo

Alertas úteis: `leme_dicas_pendentes` crescendo por mais de 5 minutos, `1 - leme_bytes_vivos/leme_bytes_disco > 0.6`, p99 de PUT acima de 50 ms e `/saude` diferente de 200. As regras prontas estão em `deploy/prometheus/alertas.yml`, e o painel do Grafana em `deploy/grafana/leme-dashboard.json`. Para uma checagem rápida, sem Prometheus, use `leme top` ou `leme doctor`.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 11

Segurança

O Leme protege o tráfego com TLS e mTLS, autentica clientes e pares com token e expõe o console só com token fora da máquina local.

11.1 Token

```
openssl rand -hex 32 > /etc/leme/token && chmod 600 /etc/leme/token
lemed ... --token-arquivo /etc/leme/token
export LEME_TOKEN=$(cat /etc/leme/token)
```

O mesmo token vale para clientes e para a conversa entre nós. Sem token, o nó só responde a `PING`. Cada tentativa errada espera 200 ms e encerra a conexão.

11.2 Certificados para TLS e mTLS

```
openssl req -x509 -newkey rsa:3072 -nodes -days 3650 -subj "/CN=leme-ca" \
    -keyout ca.key -out ca.pem
for no in no1 no2 no3; do
  openssl req -newkey rsa:3072 -nodes -subj "/CN=$no" -keyout $no.key -out $no.csr
  openssl x509 -req -in $no.csr -CA ca.pem -CAkey ca.key -CAcreateserial -days 825 -out $no.pem \
      -extfile <(printf "subjectAltName=DNS:$no,IP:10.0.0.1%s" "${no#no}")
done
openssl req -newkey rsa:3072 -nodes -subj "/CN=cli" -keyout cli.key -out cli.csr
openssl x509 -req -in cli.csr -CA ca.pem -CAkey ca.key -CAcreateserial -days 825 -out cli.pem
```

```
lemed ... --tls-cert no1.pem --tls-chave no1.key --tls-ca ca.pem
leme --nos no1:7400 --tls-ca ca.pem --tls-cert cli.pem --tls-chave cli.key cluster status
```

Com `--tls-ca` no nó, ele exige certificado de clientes e pares assinado pela CA (mTLS). Sem `--tls-ca`, o TLS só cifra o tráfego.

11.3 Console e API

O console escuta em `127.0.0.1` por padrão. Exposto, exige token, guardado em cookie `HttpOnly` e `SameSite=Strict`. Pedidos que alteram dados precisam do cabeçalho `X-Leme`, o que bloqueia formulários de outros sites, ou de `Authorization: Bearer`, que o navegador nunca envia sozinho. Todas as respostas levam Content-Security-Policy, `nosniff`, `X-Frame-Options: DENY` e `Referrer-Policy: no-referrer`. Atrás de um proxy reverso com HTTPS, mantenha o token.

Copyright (©) 2026 Erik Pimenta

11.4 Boas práticas

- Coloque os nós numa rede privada e libere a porta 7400 só para os outros nós e para quem opera.
- Use TLS sempre que o tráfego passar por redes compartilhadas.
- Não passe o token por `--token` em servidores compartilhados: ele aparece na lista de processos.
- Rode o `lemed` com um usuário próprio, sem privilégios, como fazem a imagem Docker, a unidade do systemd e o StatefulSet.
- Guarde o token do console num cofre de segredos e use-o só por `Authorization: Bearer` nas integrações.
- Monitore `/saude` e as dicas pendentes, e rode `leme doctor` depois de cada mudança no cluster.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 12

Desempenho e escala

Como medir o cluster, o que ajustar e quais são os limites de cada parte.

12.1 Medindo

```
leme bench --operacoes 50000 --concorrencia 64 --leituras 0.9 --tamanho 512
```

O bench grava as chaves de teste, mede leituras e escritas misturadas e mostra quantidade, p50, p95, p99 e máxima de cada tipo, além das operações por segundo. Em um notebook, com três nós locais e quórum, passa de 10 mil operações por segundo com 16 clientes. Acompanhe ao vivo, em outro terminal, com `leme top`. Apague as chaves de teste depois com `leme keys bench: -o texto | xargs leme del`.

12.2 O que ajustar

AJUSTE	EFEITO
`-c um` nas leituras	Menor latência; aceita ler um valor um pouco atrasado
`--sincronizar sempre`	Nenhuma escrita confirmada se perde numa queda de energia; escritas mais lentas
`--sincronizar nunca`	Escritas mais rápidas; o sistema operacional decide quando gravar
`--replicas 5`	Tolera duas quedas com quórum; mais tráfego de rede e disco
`--vnodes 256`	Distribuição mais uniforme com poucos nós; anel um pouco maior na memória
`--segmento-mb`	Segmentos menores compactam mais rápido; maiores geram menos arquivos
Mais nós	Mais capacidade total e menos chaves por nó

12.3 Limites

O índice de cada nó fica na memória: conte cerca de 100 bytes mais o tamanho da chave por registro. Um milhão de chaves de 30 bytes por nó ocupa perto de 130 MiB.

Copyright (©) 2026 Erik Pimenta

ITEM	LIMITE
Chave	64 KiB
Valor	32 MiB
Mensagem do protocolo	64 MiB
Página de scan	10.000 chaves
Conexões por nó	4.096 (ajustável)
Requisições simultâneas por conexão	256
Dicas guardadas por coordenador	100.000 chaves
Corpo de um pedido ao console (importação)	33 MiB

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 13

Testes, build e CI

Os testes sobem clusters de verdade, com nós em portas reais, e o CI repete tudo a cada push.

13.1 Comandos

COMANDO	O QUE FAZ
`go test -race ./...`	Todos os testes, com o detector de corridas
`bash scripts/ponta-a-ponta.sh bin`	Três processos `lemed` e a CLI, como no CI
`go vet ./...` e `gofmt -l .`	Análise estática e formatação
`make build`	Binários com versão e commit
`make dev`	Compila e sobe o cluster local com o console

13.2 O que os testes cobrem

ÁREA	TESTES
Armazenamento	Gravar, ler e reabrir; última escrita vence; lápides; TTL; cauda corrompida; bit trocado; compactação; varredura
Anel	Distribuição equilibrada; réplicas distintas; estabilidade ao crescer; ordem dos membros
Protocolo	Ida e volta; mensagens truncadas; quadros grandes e versão errada
Cluster	Quórum com nó derrubado; consistência `todos`; escrita condicional; TTL; token; dicas entregues ao nó que volta; saída de nó sem perda; varredura sobre lápides
CLI	Comandos diretos, JSON, códigos de saída, entrada padrão, modo não interativo, perfis, contadores, cópia e renomeação, exportação e importação de ida e volta, diagnóstico
Console	Páginas, API, `X-Leme`, token, cookie, `Authorization: Bearer`, exportar, importar, diagnóstico, OpenAPI, confirmação de comandos perigosos

13.3 Integração contínua

A cada push e pull request, o GitHub Actions confere a formatação e o `go vet`, roda os testes com o detector de corridas, compila para Linux (amd64 e arm64), Windows e macOS, sobe três processos `lemed` e exercita a CLI de ponta a ponta (incluindo a queda de um nó e os códigos de saída), e constrói a imagem Docker.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 14

Solução de problemas

Quando algo não funciona, comece por `leme doctor`: ele aponta a verificação que falhou e o que fazer. As tabelas abaixo cobrem as mensagens mais comuns.

14.1 Conexão

MENSAGEM	CAUSA PROVÁVEL	SOLUÇÃO
`não foi possível conectar a ...: connection refused`	Nó parado ou porta errada	Confira o processo e a porta; `leme ping`
`token inválido` (código 7)	Token diferente do cluster	Confira `LEME_TOKEN`, o perfil e `leme config view`
`autenticação obrigatória`	Cluster com token, CLI sem	Informe `--token` ou `--token-arquivo`
`tls: failed to verify certificate`	CA errada ou nome fora do certificado	Use a CA correta e um nome presente no `subjectAltName`
`... não respondeu a tempo` (código 4)	Rede lenta ou nó sobrecarregado	Aumente `--tempo-limite`; veja `node stats` e `leme top`
API responde 401	Token do console ausente ou errado	Envie `Authorization: Bearer <token-console>`
API responde 403 **`cabeçalho X-Leme ausente`**	Pedido de escrita sem Bearer nem `X-Leme`	Use Bearer ou acrescente `X-Leme: 1`

14.2 Dados

MENSAGEM	CAUSA PROVÁVEL	SOLUÇÃO
`quorum_nao_atingido` (código 5)	Réplicas demais fora do ar	`cluster nodes`; suba os nós ou use `-c um`
`conflito_de_versao` (código 6)	Outra escrita aconteceu antes	Leia de novo e repita com a versão nova; para contadores, use `incr`
`copy` ou **`rename`** com código 6	O destino já existe	Escolha outro nome ou use `--substituir`
Chave some depois de um tempo	TTL definido	`ttl <chave>`; `expire <chave> 0` remove
`import` com falhas	Linhas inválidas ou formato errado	Rode com `--simular` e confira `--formato`
`scan` lento	Muitas lápides acumuladas	`node compact todos`
Disco crescendo	Sobrescritas sem compactação	Acompanhe a fragmentação e compacte

Copyright (©) 2026 Erik Pimenta

14.3 Terminal

SINTOMA	SOLUÇÃO
Setas aparecem como `^[[A`	O terminal não é interativo (pipe, IDE antiga); use o Windows Terminal, o PowerShell ou outro terminal
Acentos trocados no Windows	O Leme muda o console para UTF-8 ao abrir; em terminais antigos, rode `chcp 65001` antes
Sem cores	A saída não é um terminal, `NO_COLOR` está definido ou foi usado `--sem-cor`
`top` sem desenhar a tela	A saída não é um terminal; use `cluster status` em scripts

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 15

Licenças

O Leme é código aberto, sob a licença MIT, e depende de poucos componentes de terceiros.

15.1 Componentes de terceiros

COMPONENTE	USO	LICENÇA
Cobra e pflag	Comandos e flags	Apache-2.0 e BSD-3-Clause
mousetrap	Dependência do Cobra no Windows	Apache-2.0
Instrument Sans, Bricolage Grotesque e IBM Plex Mono, via Fontsource	Tipografia do console e dos manuais	SIL Open Font License 1.1

Os textos das licenças das fontes acompanham os arquivos em `internal/console/web/fontes`. Os nomes Prometheus, Grafana e Kubernetes pertencem aos seus donos e aparecem só como ferramentas compatíveis.

15.2 Licença do Leme

Copyright (©) 2026 Erik Pimenta

O Leme é distribuído sob a licença MIT. Você pode usar, copiar, modificar e redistribuir o código, inclusive para fins comerciais, desde que mantenha o aviso de copyright e o texto da licença, que está no arquivo `LICENSE`.